PROYECTO DE LA UNIDAD I

Desarrollo de un Custom Agent para Simulación Explicable de Circuitos

Netlist → Grafo → Matrices → Análisis AC → Ganancia → Validación

Reto central

Completar las herramientas faltantes, integrar los 10 skills y construir un agente de GitHub Copilot capaz de analizar el filtro activo mostrado, obtener su respuesta en frecuencia y comparar magnitud y fase contra un simulador SPICE.

Profesor: Dr. Francisco E. Rangel-Patiño

Matemáticas Avanzadas · Programa 2026
Maestría en Ingeniería Electrónica
Tecnológico Superior de Jalisco

1. Propósito del proyecto

El proyecto integra álgebra lineal, teoría de grafos, formulación nodal, Tableau, análisis fasorial y desarrollo de agentes. El resultado final será un agente educativo que reciba un netlist, construya los artefactos matemáticos del circuito, resuelva el análisis AC y explique cómo obtuvo cada resultado.

Resultado esperado

El agente no debe funcionar como una caja negra. Cada voltaje, corriente, matriz, gráfica y métrica de validación deberá conservar trazabilidad hasta el netlist y las ecuaciones constitutivas.

2. Circuito objetivo

El circuito objetivo es un filtro activo con dos capacitores de entrada, una red de realimentación mediante G_1, una conductancia G_2 a tierra y un amplificador operacional no inversor con resistores R_A y R_B. El agente deberá modelar el amplificador operacional ideal mediante la formulación aprobada en el curso.

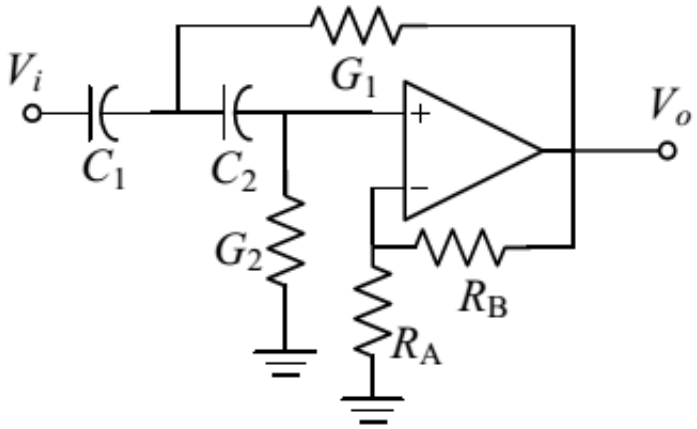


Figura 1. Diagrama conceptual del circuito objetivo.

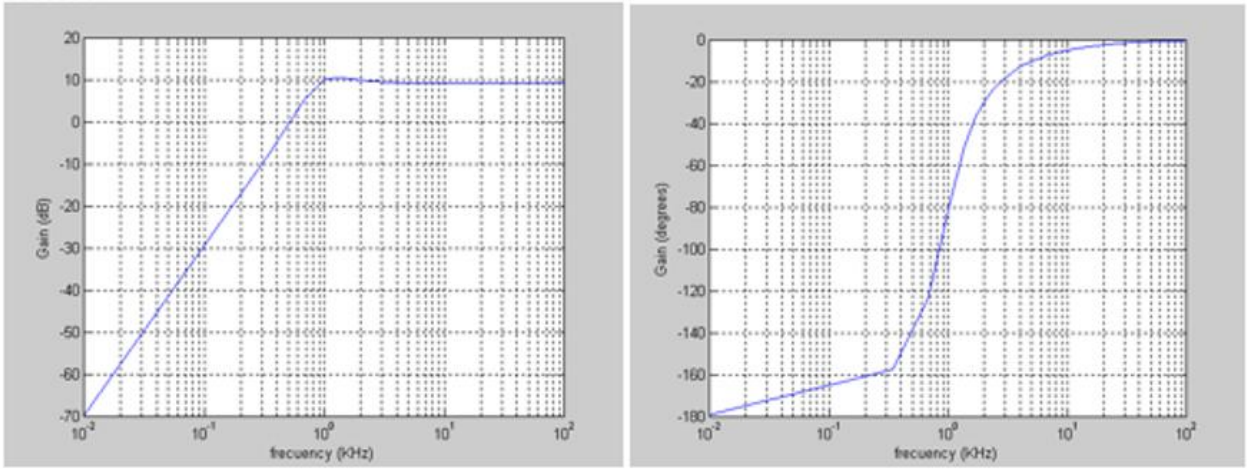
Los valores de C_1, C_2, G_1, G_2, R_A, R_B, la excitación V_i y el barrido de frecuencia deberán definirse en el netlist oficial del proyecto. El agente no deberá codificar estos valores dentro de los scripts.

3. Referencia del resultado esperado

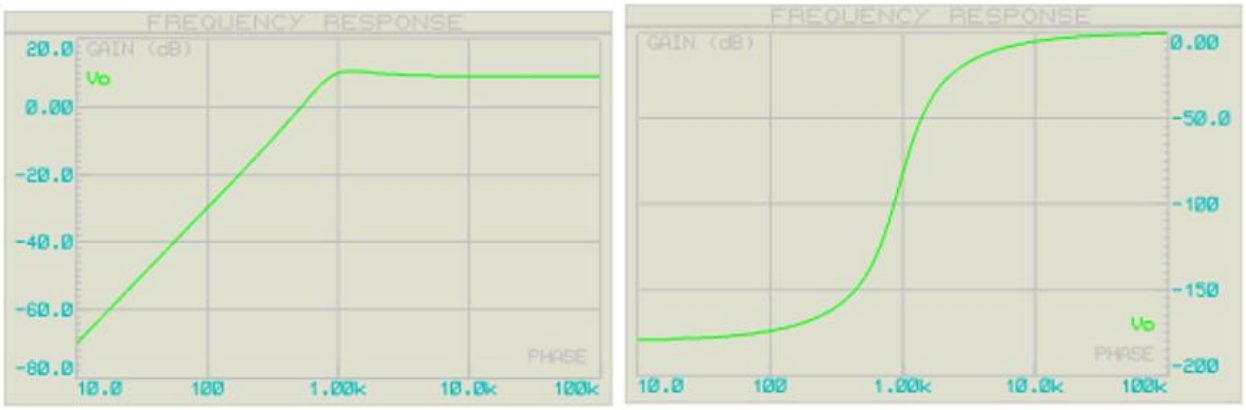
La salida final deberá incluir dos gráficas de Bode: ganancia en decibeles y fase en grados. La imagen anexa muestra el tipo de comparación requerida entre la implementación matemática y el simulador SPICE.

Prueba de Concepto

Resultados de la formulación nodal en MATLAB (ganancia) se comparan con los obtenidos mediante el simulador de circuitos SPICE.



Implementación de la formulación nodal en MATLAB (ganancia): magnitud y fase.



Simulador de circuitos SPICE (ganancia): magnitud y fase.

Figura 2. Referencia visual para la comparación de magnitud y fase.

4. Objetivos de aprendizaje

- Aplicar teoría de grafos y matrices a un circuito electrónico real.
- Convertir un netlist en un modelo computacional reproducible.
- Construir matrices de incidencia y formulaciones de circuito.
- Implementar análisis nodal, Tableau y análisis AC con NumPy.
- Diseñar tools deterministas para un Custom Agent de GitHub Copilot.
- Aplicar validación matemática, eléctrica y numérica.
- Comparar resultados contra un simulador SPICE y comunicar diferencias.

5. Requerimientos funcionales del agente

1. Recibir como entrada un netlist compatible con el subconjunto documentado.
2. Validar sintaxis, unidades, nodo de referencia y componentes soportados.
3. Identificar componentes, nodos, ramas y orientación.
4. Generar y mostrar el grafo orientado.
5. Generar la matriz de incidencia completa y reducida con encabezados.

6. Construir las matrices constitutivas requeridas por el método seleccionado.
7. Modelar las ramas especiales asociadas con la fuente de entrada y el amplificador operacional ideal.
8. Ensamblar y resolver el sistema lineal para cada frecuencia.
9. Calcular $H(j\omega)=V_o/V_i$.
10. Calcular ganancia en dB y fase en grados.
11. Generar gráficas comparables con SPICE.
12. Validar residuos, KCL, KVL y ecuaciones constitutivas.
13. Generar un reporte explicable y reproducible.

6. Cálculos obligatorios

$$H(j\omega) = Vo(j\omega) / Vi(j\omega)$$
$$Ganancia_dB = 20 \log10(|H(j\omega)|)$$
$$Fase_deg = angle(H(j\omega)) \cdot 180 / \pi$$

La fase deberá desenvolverse cuando sea necesario para evitar saltos artificiales de ±180°. Las gráficas deberán usar frecuencia en escala logarítmica, incluir unidades, títulos, leyendas y etiquetas claras.

Resultado	Contenido mínimo
Grafo	Nodos, ramas, orientación, identificador de componente.
Matriz A	Forma completa
Matrices de análisis	Y, Z, W y/o bloques MNA, según el método implementado.
Respuesta AC	V_i, V_o, $H(j\omega)$, magnitud, dB y fase.
Validación	Residuo, KCL, KVL, constitutivas, rango y condición.
Comparación	Curvas propias y SPICE en condiciones equivalentes.

7. Trabajo proporcionado y trabajo por desarrollar

7.1 Material proporcionado por el profesor

Paquete	Contenido	Uso
Guia_Metodologica_Custom_Agent_SPICE_Educativo.md	Metodología completa del agente: arquitectura, responsabilidades del agente/skills/tools, convenciones, flujo determinista, validación, pruebas y definicion de “done”.	Documento rector del proyecto.
PLAN_01_Desarrollo_9_Scripts_basic-spice-agent.md	Prompt de PLAN para analizar el repositorio, definir dependencias, interfaces, pruebas y estrategia de implementación de los nueve scripts Python restantes.	Guía de planeación para desarrollar las tools faltantes.
PLAN_02_Integracion_basic-spice-agent.md	Prompt de PLAN para auditar e integrar los 15 tools y 10 skills, definir la orquestación del agente, AC sweep, validación, comparación con SPICE y calificación final con el circuito de Unidad I.	Guía de integración y construcción final del Custom Agent.
SPICE_Educational_Agent_10_Skills.zip	Diez skills con conocimiento especializado sobre netlists, grafos, matrices de incidencia, modelos constitutivos, análisis nodal, Tableau, AC, convenciones, validación y reporting. El proyecto exige integrar estos 10 skills.	Base de conocimiento y metodología del agente
SPICE_Educational_Agent_First_6_Python_Tools.zip	Seis tools (Python scripts) iniciales: parse_netlist.py, build_circuit_model.py, build_oriented_graph.py, build_incidence_matrix.py, build_nodal_system.py y solve_linear_system.py.	Base ejecutable.

7.2 Primeros seis scripts entregados

#	Script	Responsabilidad
1	parse_netlist.py	Interpretar el subconjunto educativo del netlist.
2	build_circuit_model.py	Crear nodos, ramas y objetos del dominio.
3	build_oriented_graph.py	Construir y exportar el grafo dirigido.
4	build_incidence_matrix.py	Generar A completa y reducida.
5	build_nodal_system.py	Ensamblar el sistema nodal compatible.
6	solve_linear_system.py	Resolver, recuperar variables y validar KCL.

7.3 Nueve scripts que deberán desarrollar los estudiantes

#	Script requerido	Responsabilidad principal
1	validate_netlist.py	Validación detallada, errores y alcance soportado.
2	stamp_elements.py	Stamps deterministas de elementos lineales y ramas especiales.
3	build_constitutive_matrices.py	Construcción trazable de Y, Z y W.
4	build_tableau_system.py	Ensamble de T, W' y mapas de índices.
5	solve_ac_sweep.py	Barrido de frecuencia y extracción de H(jω).
6	validate_solution.py	Validación matemática, eléctrica y numérica.
7	compare_reference.py	Comparación cuantitativa contra resultados SPICE.
8	generate_report.py	Reporte explicable en Markdown/JSON.
9	plot_results.py	Gráficas de ganancia en dB y fase en grados.

8. Skills que deberá integrar el agente

#	Skill
1	spice-netlist-fundamentals
2	circuit-graph-theory
3	incidence-matrix-method
4	component-constitutive-models
5	classical-nodal-analysis
6	tableau-formulation
7	ac-frequency-analysis
8	engineering-units-and-conventions
9	solver-validation
10	explainable-circuit-reporting

Los skills constituyen el conocimiento y la metodología. Los scripts constituyen las tools ejecutables. El agente deberá seleccionar ambos en el orden apropiado y no calcular matrices únicamente mediante generación de texto.

9. Flujo de trabajo esperado del agente

Netlist
↓
Validación sintáctica y de alcance
↓
Modelo del circuito y grafo orientado
↓
Matriz de incidencia
↓
Modelos constitutivos y ramas especiales
↓
Sistema nodal / Tableau / MNA
↓
Barrido AC y solución con NumPy
↓
H(jω), ganancia dB y fase degrees
↓
Validación y comparación con SPICE
↓
Reporte explicable

10 Procedimiento para completar el basic-spice-agent

Primero deberá completarse y validarse el motor determinista de simulación. La arquitectura establece que el agente interpreta, selecciona el flujo y explica resultados; los skills contienen conocimiento y metodología; y las herramientas Python realizan los cálculos deterministas.

Paso 1. Preparar el repositorio

Organice el repositorio utilizando como referencia la **Guía Metodológica** y coloque en él:

- Los 10 skills proporcionados.
- Los 6 scripts Python iniciales.
- Los tres documentos Markdown proporcionados por el profesor.

La guía metodológica propone separar agente, prompts, skills, código fuente, pruebas, ejemplos y artefactos.

Paso 2. Estudiar el material proporcionado

Antes de modificar código:

1. Revise “Guia_Metodologica_Custom_Agent_SPICE_Educativo”.
2. Revise los 10 SKILL.md.
3. Estudie los 6 scripts Python proporcionados.
4. Comprenda cómo se representan nodos, ramas, orientación, unidades y matrices.

Las convenciones compartidas incluyen unidades, nodo de tierra, polaridades, orientación de ramas, orden de nodos y ramas, signos matriciales, números complejos y tolerancias.

Paso 3. Utilizar PLAN_01 antes de programar

Utilice “PLAN_01_Desarrollo_9_Scripts_basic-spice-agent” con GitHub Copilot en **Plan Mode**.
En esta etapa **no deberá generar todavía los nueve scripts**. El objetivo es que Copilot inspeccione el repositorio y proponga:

- Dependencias.
- Interfaces.
- Entradas y salidas.
- Algoritmos.
- Contratos de datos.
- Pruebas unitarias.
- Pruebas de integración.
- Criterios de terminación.

El PLAN solicita explícitamente a Copilot analizar primero los seis scripts existentes y mantener compatibilidad con la arquitectura establecida.
Resultado de este paso: Plan de Desarrollo aprobado para los nueve scripts.

Paso 4. Desarrollar los nueve scripts Python restantes

A partir del Plan aprobado, implemente:

1. validate_netlist.py
2. stamp_elements.py
3. build_constitutive_matrices.py
4. build_tableau_system.py
5. solve_ac_sweep.py
6. validate_solution.py
7. compare_reference.py
8. generate_report.py
9. plot_results.py

Estos scripts cubren validación, estampado de componentes, construcción de Y, Z, W, Tableau, barrido AC, validación, comparación con SPICE, reporte y gráficas.
Resultado de este paso:

$$6 \text{ tools proporcionados} + 9 \text{ tools desarrollados} = \boxed{15 \text{ tools (Python Scripts)}}$$

Paso 5. Utilizar PLAN_02 para diseñar la integración

Una vez terminadas las 15 tools, utilice “PLAN_02_Integracion_basic-spice-agent” con GitHub Copilot en **Plan Mode**.
Copilot deberá auditar:

- Los 15 scripts.
- Los 10 skills.
- Contratos de datos.
- Convenciones.
- Interfaces.
- Dependencias.
- Tests.
- Flujo AC.
- Validaciones.
- Integración con SPICE.

El PLAN requiere construir explícitamente el mapeo **Skill → Tool(s)** y revisar las interfaces antes de implementar el agente.
Resultado de este paso: Plan de Integración aprobado.

Paso 6. Corregir los gaps encontrados

Antes de crear el Custom Agent, resuelva cualquier problema identificado por PLAN_02, por ejemplo:

- Interfaces incompatibles.
- Diferencias en orden de nodos o ramas.
- Inconsistencias de signos.
- Falta de metadatos.
- Tests faltantes.
- Problemas con el OpAmp ideal.
- Inconsistencias en $H(j\omega)$, dB o fase.
-

Paso 7. Construir el basic-spice-agent

Proceda a implementar el plan y desarrollar el Agente:

```
.github/
└── agents/
└── basic-spice-agent.agent.md
```

El agente deberá:

- Interpretar la solicitud.
- Seleccionar los skills necesarios.
- Seleccionar las tools correspondientes.
- Ejecutarlas en el orden apropiado.
- Detenerse ante errores bloqueantes.
- Validar los resultados.
- Explicar el procedimiento.
- Generar los artefactos solicitados.

El agente **no deberá calcular manualmente** matrices, soluciones del sistema, voltajes, corrientes, $H(j\omega)$, ganancia o fase. Estas operaciones pertenecen a las tools deterministas.

Paso 8. Calificar el agente con el circuito de Unidad I

Finalmente entregue el netlist oficial del circuito objetivo al **basic-spice-agent**.
El circuito contiene dos capacitores de entrada, la realimentación G_1, G_2a tierra y un amplificador operacional no inversor con R_Ay R_B.

El agente deberá producir un reporte que incluya:

- Netlist.
- Componentes.
- Nodos.
- Ramas.
- Grafo.
- Matriz de incidencia.
- Formulación seleccionada.
- Matrices con etiquetas.
- Resultados.
- Ganancia y fase.
- Validación.
- Comparación SPICE.
- Advertencias.
- Limitaciones.
- Información necesaria para reproducir la ejecución.
-

Paso 9. Comparar contra SPICE

Ejecute el circuito equivalente en SPICE utilizando las mismas:

- Topología.
- Fuentes.
- Componentes.
- Condiciones de análisis.
- Frecuencias.

Compare magnitud y fase cuantitativamente. El proyecto exige que esta comparación use condiciones equivalentes.

11. Validación contra SPICE

La comparación deberá realizarse utilizando el mismo netlist equivalente, valores de componentes, modelo ideal del amplificador operacional, amplitud de entrada y barrido de frecuencia.

Métrica	Requisito
Magnitud	Error absoluto y relativo en puntos seleccionados.
Fase	Diferencia angular en grados con tratamiento consistente de wrapping.
Frecuencia característica	Comparación del punto de transición identificado por ambos métodos.
Ganancia de banda	Comparación del valor en la región estable.
Residuos	Evidencia de que el sistema matemático satisface las tolerancias.

12. Entregables

Entregable	Contenido
Repositorio Git	Código, agent, 10 skills, prompts, pruebas y documentación.
Nueve scripts	Implementación completa con encabezado, autor, docstrings y CLI.
Netlist final	Circuito objetivo y configuración del barrido AC.
Reporte técnico	Matemáticas, arquitectura, resultados, validación y limitaciones.
Demo del agente	Ejecución desde netlist hasta reporte final.
Comparación SPICE	Datos, gráficas y métricas de diferencia.
Presentación final	Resumen ejecutivo y demostración técnica.

13. Estructura mínima del reporte final

14. Portada y resumen ejecutivo.
15. Definición del problema y circuito.
16. Arquitectura del agente.
17. Netlist.
18. Grafo orientado.
19. Matriz de incidencia.
20. Matrices (Admitancia, Impedancia) requeridas por el método seleccionado.
21. Formulación matemática seleccionada.
22. Implementación de los nueve scripts.
23. Respuesta AC: $H(j\omega)$, dB y fase.
24. Validación matemática y numérica.
25. Comparación contra SPICE.
26. Limitaciones, y conclusiones.

14. Política de uso de GitHub Copilot

GitHub Copilot podrá utilizarse para planear, implementar, revisar, probar y documentar. Sin embargo, el estudiante es responsable de comprender y defender las ecuaciones, matrices, algoritmos y decisiones. El uso de IA no sustituye la validación matemática ni la revisión humana.